then scatter it throughout your application. It's obvious that this scenario is creating a maintenance headache. One day you are asked to make a change that affects every email sent by the application. You have a pretty bad day. Then the bug reports start arriving.

Now imagine that, rather than repeating the mailing code every time the application needed it, you wrote it in one place and just called that code every time. Making a change affecting all emails now takes you half an hour, after which you can just get some coffee. You have a much better day, and there are (probably) no bug reports. This is why we should strive for DRY code. (In case you're wondering, the repeating email scenario isn't something I made up – it was a real problem I had to fix for a client.)

In this chapter, we will examine the following topics:

- Declaring Functions
- Calling Functions
- Named Functions
- Parameters and Arguments
- Data Types and Type Declarations
- Anonymous Functions
- Variable Functions
- Lexical Scope
- Namespacing
- Functional Programming

Declaring and Calling a Named Function

To declare a named function, use the `function` keyword, followed by the name of the function, then parentheses, then the function code enclosed in braces. Functions that have names (and not all functions do; more on this to come) must follow the same naming rules as variables: Start with a letter or underscore followed by any number of letters, underscores, or digits. Any alphabetic character of the extended ASCII character set is acceptable. The same guidance for variable naming is generally applicable to functions. To call the function, we simply write its name followed by parentheses. It is the presence of parentheses that tells the PHP engine it is dealing with a function: